

Trabajo Práctico 3

Procesamiento de Imágenes y Visión por Computadora

6 de Noviembre de 2025

Punto 1: Detector de Canny

John F. Canny (1986)

Idea General

El detector de Canny es óptimo bajo la condición de ruido Gaussiano, su intención es mejorar los detectores ya existentes. Cumple con tres criterios fundamentales para una detección robusta de bordes.

Criterios:

- 1 **Buena detección:** Minimizar falsos positivos y negativos, logrando una detección de bordes más fiel.
- 2 **Buena localización:** Los bordes detectados debe de estar cerca de los reales.
- 3 **Respuesta simple:** Se devuelve un único punto de borde, sin bordes gruesos o discontinuados.

Etapas de Implementación del Detector de Canny

Etapas 1

- **1. Suavizado y derivación:** Se aplica un **filtro Gaussiano** para reducir el ruido y suavizar la imagen antes de calcular el gradiente:

Ecuación del filtro Gaussiano

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Luego se obtiene la imagen suavizada:

$$I_G(x, y) = I(x, y) * G(x, y)$$

Etapas 1

```
1 def bordes_canny():
2     global imagen_actual, imagen_operativa, umbral1, umbral2, sigma_var,
        roi_actual
3
4
5     imagen_original = imagen_actual.copy().astype(np.float32)
6     sigma = sigma_var
7     t = int((2*sigma)+1)
8     ax = np.linspace(-(t - 1) / 2, (t - 1) / 2, t)
9     xx, yy = np.meshgrid(ax, ax)
10    kernel_init = np.exp(-(xx**2 + yy**2) / (2 * sigma**2))
11
12    if len(imagen_original.shape) == 3:
13        imagen_original = cv2.cvtColor(imagen_original, cv2.COLOR_BGR2GRAY
        )
14
15    imagen_ruidosa = fc.mascara(imagen_original, kernel_init, "
        Gaussiano", grises=True, estandarizar=True)
16
```

Etapas de Implementación del Detector de Canny

Etapas 2

- **2. Cálculo del gradiente:** Se utilizan operadores de Sobel para obtener las derivadas parciales:

Componentes del gradiente

$$I_x = \frac{\partial I_G}{\partial x}, \quad I_y = \frac{\partial I_G}{\partial y}$$

La **magnitud del gradiente** se calcula como:

$$|\nabla I| = \sqrt{I_x^2 + I_y^2}$$

donde la dirección del gradiente es **perpendicular al borde**.

Etapa 2

```
1  #sobel_horizontal = fc.mascara(imagen_ruidosa, mascara=  
    mascara_horizontal, tipo_kernel="Sobel Horizontal",grises=True)  
2  #sobel_vertical = fc.mascara(imagen_ruidosa, mascara=mascara_vertical,  
    tipo_kernel="Sobel Vertical",grises=True)  
3  
4  sobel_x = cv2.Sobel(imagen_ruidosa, cv2.CV_64F, 1, 0, ksize=3)  
5  ## El problema estaba en Sobel??  
6  # Aplicar el filtro Sobel en la direccin Y (vertical)  
7  sobel_y = cv2.Sobel(imagen_ruidosa, cv2.CV_64F, 0, 1, ksize=3)  
8  
9  magnitud = np.hypot(sobel_x,sobel_y)  
10
```

Etapas de Implementación del Detector de Canny

Etapas 3

- **3. Cálculo del ángulo del gradiente (θ):** Si el gradiente horizontal $I_x = 0$, el ángulo se define como $\frac{\pi}{2}$; de lo contrario:

Dirección del gradiente

$$\theta = \arctan \left(\frac{I_y}{I_x} \right)$$

Etapa 3

```
1 direccion = np.arctan2(sobel_y, sobel_x)
2
3 direccion = np.degrees(direccion)
4
5
6 magnitud_norm = cv2.normalize(magnitud, None, alpha=0, beta=255,
7                               norm_type=cv2.NORM_MINMAX)
8 magnitud_norm = magnitud_norm.astype(np.float32)
```


Etapas de Implementación del Detector de Canny

Etapas 4

- **4. Cuantización de la dirección del gradiente (ϕ):** Se aproxima θ a una de las cuatro direcciones principales:

$$\phi \in \{0^\circ, 45^\circ, 90^\circ, 135^\circ\}$$

Cuantización de θ en ϕ

$$\phi = \begin{cases} 0^\circ, & \text{si } \theta \in [0^\circ, 22,5^\circ) \cup [157,5^\circ, 180^\circ) \\ 45^\circ, & \text{si } \theta \in [22,5^\circ, 67,5^\circ) \\ 90^\circ, & \text{si } \theta \in [67,5^\circ, 112,5^\circ) \\ 135^\circ, & \text{si } \theta \in [112,5^\circ, 157,5^\circ) \end{cases}$$

Etapas 4

```
def bordes_canny(magnitud,direccion, umbral1=100, umbral2=200):  
  
    h, w = magnitud.shape[:2]  
    Z = np.zeros((h,w), dtype=np.float32)  
    sectors = np.zeros_like(direccion, dtype=np.uint8)  
    sectors[((direccion > -22.5) & (direccion <= 22.5)) |  
            ((direccion <= -157.5) | (direccion > 157.5))] = 0  
  
    # Sector 45: diagonal  
    sectors[((direccion > 22.5) & (direccion <= 67.5)) |  
            ((direccion <= -112.5) & (direccion > -157.5))] = 45  
  
    # Sector 90: vertical  
    sectors[((direccion > 67.5) & (direccion <= 112.5)) |  
            ((direccion <= -67.5) & (direccion > -112.5))] = 90  
  
    # Sector 135: diagonal  
    sectors[((direccion > 112.5) & (direccion <= 157.5)) |  
            ((direccion <= -22.5) & (direccion > -67.5))] = 135
```

Etapas de Implementación del Detector de Canny

Etapas 5

- **5. Supresión de no máximos:** Esta etapa elimina píxeles que no son máximos locales en la dirección del gradiente. Se compara cada píxel con sus dos vecinos a lo largo de ϕ :

Condición de máximo local

$$M(x, y) = \begin{cases} M(x, y), & \text{si } M(x, y) \geq M(x_1, y_1) \text{ y } M(x, y) \geq M(x_2, y_2) \\ 0, & \text{en otro caso} \end{cases}$$

donde $M(x, y)$ es la magnitud del gradiente y (x_1, y_1) , (x_2, y_2) son los vecinos en la dirección de ϕ .

Aporte Clave

La **supresión de no máximos** es la etapa que garantiza bordes delgados y precisos, mientras que la **histéresis** evita rupturas en la continuidad de los bordes.

Etapas 5

```
1 for i in range(1, h-1):
2     for j in range(1, w-1):
3         val = magnitud[i, j]
4         sector = sectores[i, j]
5
6         if sector == 0:      # Horizontal
7             if val >= magnitud[i, j+1] and val >= magnitud[i, j-1]:
8                 Z[i, j] = val
9         elif sector == 45:   # Diagonal
10            if val >= magnitud[i-1, j+1] and val >= magnitud[i+1, j
11-1]:
12                Z[i, j] = val
13        elif sector == 90:   # Vertical
14            if val >= magnitud[i-1, j] and val >= magnitud[i+1, j]:
15                Z[i, j] = val
16        else:                # 135 Diagonal
17            if val >= magnitud[i-1, j-1] and val >= magnitud[i+1, j
18+1]:
19                Z[i, j] = val
```

Etapas de Implementación del Detector de Canny

Etapas 6

- **6. Umbralización con histéresis:** Se aplican dos umbrales $t_1 < t_2$ para distinguir bordes fuertes y débiles:

Regla de histéresis

$$E(x, y) = \begin{cases} 1, & \text{si } M(x, y) \geq t_2 \quad (\text{borde fuerte}) \\ 1, & \text{si } t_1 \leq M(x, y) < t_2 \text{ y está conectado a un borde fuerte} \\ 0, & \text{si } M(x, y) < t_1 \quad (\text{no borde}) \end{cases}$$

Aporte Clave

La **supresión de no máximos** es la etapa que garantiza bordes delgados y precisos, mientras que la **histéresis** evita rupturas en la continuidad de los bordes.

Etapa 5

```
1 Z[Z < umbral1] = 0
2 Z[Z >= umbral2] = 255
3 mid = (Z >= umbral1) & (Z < umbral2)
4 Z[mid] = 128
5 cambios = True
6 while cambios:
7     cambios = False
8     for i in range(1, h-1):
9         for j in range(1, w-1):
10             if Z_copy[i,j] == 128:
11                 vecinos = [Z_copy[i-1,j], Z_copy[i+1,j], Z_copy[i,j
12                 -1], Z_copy[i,j+1],
13                 Z_copy[i-1,j-1], Z_copy[i-1,j+1], Z_copy[i+1,j-1],
14                 Z_copy[i+1,j+1]]
15                 if any(v == 255 for v in vecinos):
16                     Z_copy[i,j] = 255
17                     cambios = True
18             else:
19                 Z_copy[i,j] = 0
```

Evaluación Canny



Figura: Imagen Original

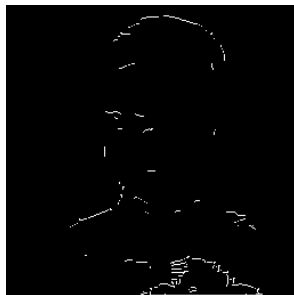


Figura: Umbral1 = 20,
Umbral2 = 100



Figura: Umbral1 = 20,
Umbral2 = 39

Evaluación Canny

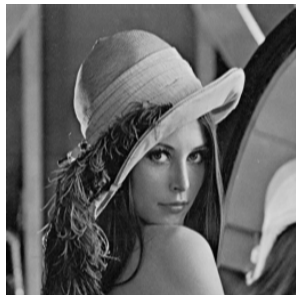


Figura: Imagen Original



Figura: Umbral1 = 20,
Umbral2 = 100



Figura: Umbral1 = 26,
Umbral2 = 52

Evaluación Canny



Figura: Imagen Original



Figura: Umbral1 = 20,
Umbral2 = 100



Figura: Umbral1 = 30,
Umbral2 = 59

Evaluación Canny Ruidosa Gauss



Figura: Imagen Ruidosa, 10 % de la imagen con 5 desviación.



Figura: Umbral1 = 20, Umbral2 = 100



Figura: Umbral1 = 26, Umbral2 = 51

Evaluación Canny Ruidosa Gauss



Figura: Imagen Ruidosa, 30 % de la imagen con 10 desviación.



Figura: Umbral1 = 20, Umbral2 = 100



Figura: Umbral1 = 26, Umbral2 = 51

Evaluación Canny Ruidosa Gauss



Figura: Imagen Ruidosa, 50 % de la imagen con 20 desviación.



Figura: Umbral1 = 20, Umbral2 = 100



Figura: Umbral1 = 28, Umbral2 = 55

Evaluación Canny Ruidosa Exponencial



Figura: Imagen Ruidosa,
10 % de la imagen con
0.01.



Figura: Umbral1 = 20,
Umbral2 = 100

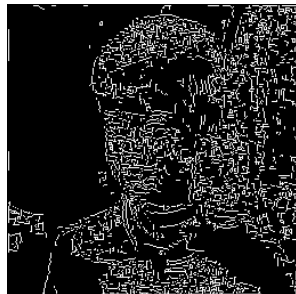


Figura: Umbral1 = 22,
Umbral2 = 43

Evaluación Canny Ruidosa Exponencial



Figura: Imagen Ruidosa, 30 % de la imagen con 0.05.



Figura: Umbral1 = 20, Umbral2 = 100



Figura: Umbral1 = 24, Umbral2 = 47

Evaluación Canny Ruidosa Exponencial



Figura: Imagen Ruidosa,
50 % de la imagen con
0.1.

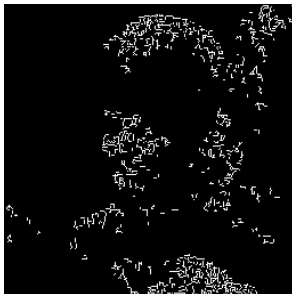


Figura: Umbral1 = 20,
Umbral2 = 100



Figura: Umbral1 = 24,
Umbral2 = 47

Evaluación Canny Ruidosa Sal y Pimienta



Figura: Imagen Ruidosa con 0.01.



Figura: Umbral1 = 20, Umbral2 = 100

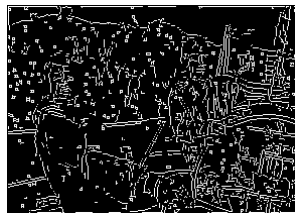


Figura: Umbral1 = 30, Umbral2 = 60

Evaluación Canny Ruidosa Sal y Pimienta



Figura: Imagen Ruidosa con 0.05.

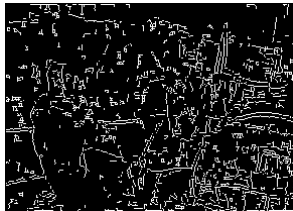


Figura: Umbral1 = 20, Umbral2 = 100

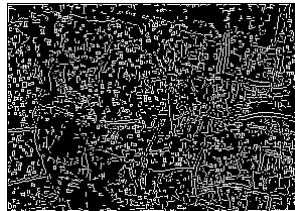


Figura: Umbral1 = 32, Umbral2 = 64

Evaluación Canny Ruidosa Sal y Pimienta



Figura: Imagen Ruidosa con 0.1.



Figura: Umbral1 = 20, Umbral2 = 100

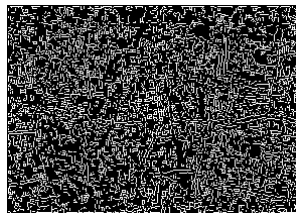


Figura: Umbral1 = 36, Umbral2 = 75

Punto 2: Detector SUSAN

SUSAN usa una máscara circular (≈ 37 píxeles) para comparar niveles de gris con un núcleo r_0 .

Algoritmo:

- 1 Contar los píxeles similares al núcleo dentro de un umbral t .
- 2 Para ello

$$c(r, r_0) = \begin{cases} 1, & \text{si } |I(r) - I(r_0)| < t \\ 0, & \text{si } |I(r) - I(r_0)| \geq t \end{cases}$$

- 3 Calcular $n(r_0) = \sum_{i=1}^{36} c(r, r_0)$
- 4 Calcular $s(r_0) = 1 - \frac{n(r_0)}{37}$.

Interpretación:

- $s(r_0) \approx 0$: área homogénea.
- $s(r_0) \approx 0,5$: borde.
- $s(r_0) \approx 0,75$: esquina.

General

```
1 def susan_bordes():
2     global imagen_actual, imagen_operativa, umbral_susan, tipo_susan
3
4     imagen_original = imagen_actual.copy().astype(np.float32)
5     if len(imagen_original.shape) == 3:
6         imagen_original = cv2.cvtColor(imagen_original, cv2.
7             COLOR_BGR2GRAY)
8     bordes = fc.susan_bordes(imagen_original, umbral=umbral_susan,
9         tipo=str(tipo_susan))
10    imagen_operativa = bordes
11    mostrar_imagen(imagen_operativa, lblOutputImage)
12    set_status(f"Bordes Susan aplicado a la imagen de trabajo.")
```

Paso 1, 2 y 3

```
1 def susan_bordes(imagen, umbral=15, tipo="borde"):  
2     H, W = imagen.shape  
3     imagen_susan = np.zeros_like(imagen, dtype=np.uint8)  
4  
5     for i in range(H):  
6         for j in range(W):  
7             centro = imagen[i, j]  
8             suma = 0  
9  
10            for di in range(-3, 4):  
11                for dj in range(-3, 4):  
12                    if di*di + dj*dj <= 9:  
13                        x, y = i + di, j + dj  
14  
15                        if 0 <= x < H and 0 <= y < W:  
16                            diff = abs(imagen[x, y] - centro)  
17                            if diff < umbral:  
18                                suma += 1  
19
```

Paso 4

```
1 sr = 1 - suma/37
2 if sr < 0.35:
3     imagen_susan[i, j] = 0
4 elif sr > 0.35 and sr < 0.65:
5     imagen_susan[i, j] = 1
6 elif sr >= 0.65:
7     imagen_susan[i, j] = 2
8
9 imagen_susan = mostrar_tipo_susan(imagen_susan, imagen, tipo)
10
11 return imagen_susan
12
```

Paso 3

```
1 def mostrar_tipo_susan(imagen_susan, imagen_original, tipo):
2     imagen_bgr = cv2.cvtColor(imagen_original, cv2.COLOR_GRAY2BGR)
3
4     # Crear mscara segn tipo
5     if tipo == "borde":
6         mascara = imagen_susan == 1
7     elif tipo == "esquina":
8         mascara = imagen_susan == 2
9     elif tipo == "ambos":
10        mascara = (imagen_susan == 1) | (imagen_susan == 2)
11
12    # Pintamos sobre la imagen original en verde
13    imagen_bgr[mascara] = [0, 0, 255] # rojo slido
14
15    return imagen_bgr.astype(np.uint8)
16
```

Evaluación Susan

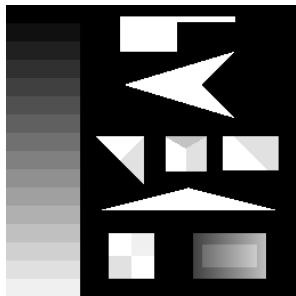


Figura: Imagen Original

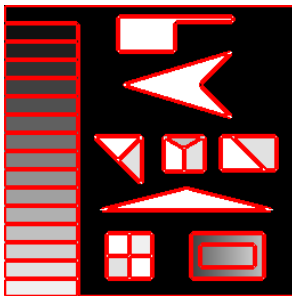


Figura: Bordes

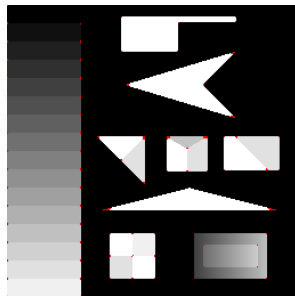


Figura: Esquinas

Evaluación Susan Ruidosa Gauss

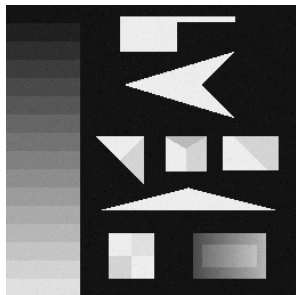


Figura: Imagen Ruidosa, 10 % de la imagen con 5 desviación.

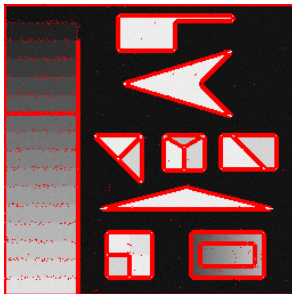


Figura: Bordes

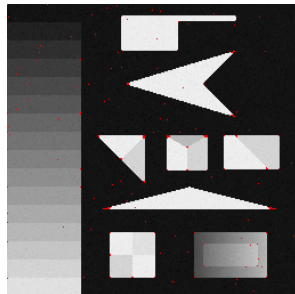


Figura: Esquinas

Evaluación Susan Ruidosa Gauss

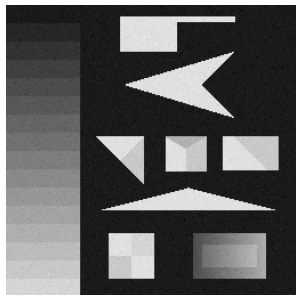


Figura: Imagen Ruidosa,
30 % de la imagen con 10
desviación.

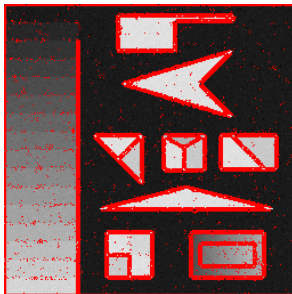


Figura: Bordes

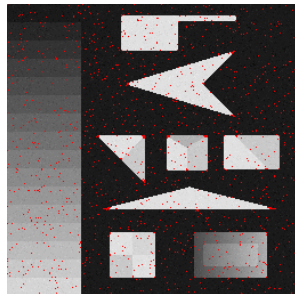


Figura: Esquinas

Evaluación Susan Ruidosa Gauss



Figura: Imagen Ruidosa,
50 % de la imagen con 20
desviación.

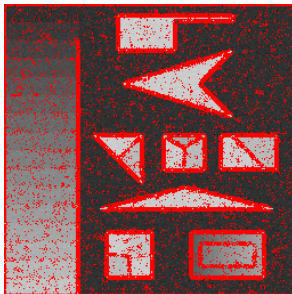


Figura: Bordes

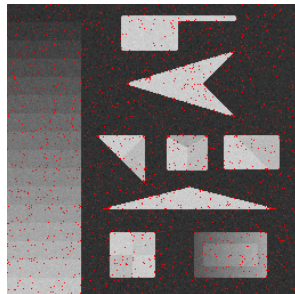


Figura: Esquinas

Evaluación Susan Ruidosa Exponencial

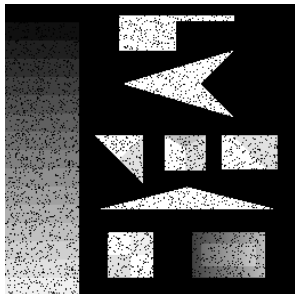


Figura: Imagen Ruidosa,
10 % de la imagen con
0.01.

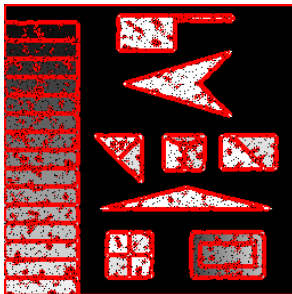


Figura: Borde

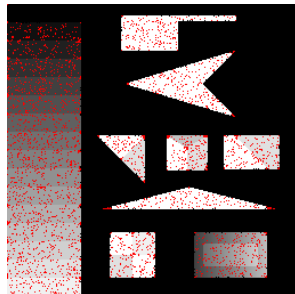


Figura: Esquina

Evaluación Susan Ruidosa Exponencial

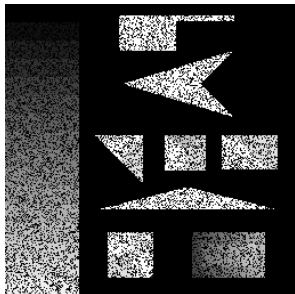


Figura: Imagen Ruidosa,
30 % de la imagen con
0.05.



Figura: Borde

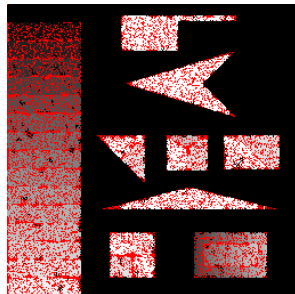


Figura: Esquina

Evaluación Susan Ruidosa Exponencial

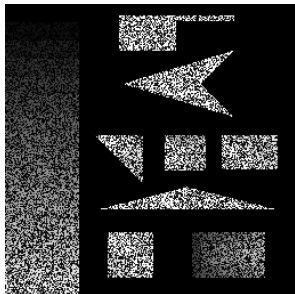


Figura: Imagen Ruidosa,
50 % de la imagen con
0.1.

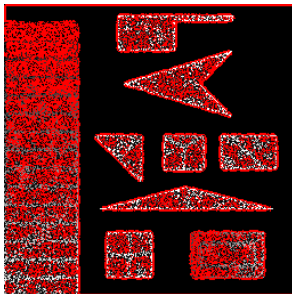


Figura: Borde

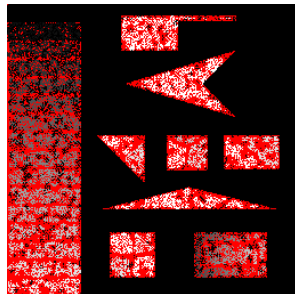


Figura: Esquina

Evaluación Susan Ruidos Sal y Pimienta

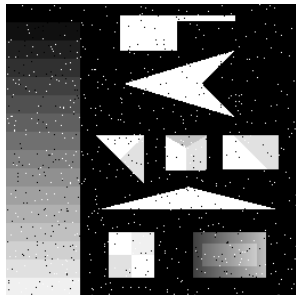


Figura: Imagen Ruidosa con 0.01.

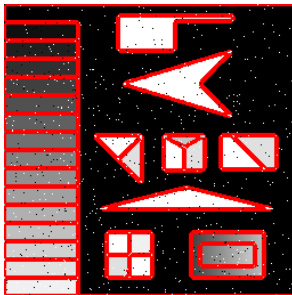


Figura: Borde

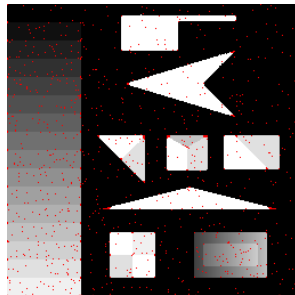


Figura: Esquina

Evaluación Susan Ruidos Sal y Pimienta

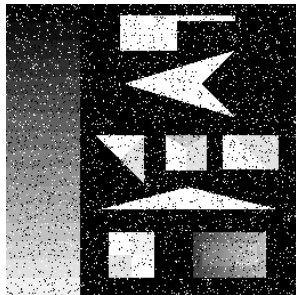


Figura: Imagen Ruidosa con 0.05.

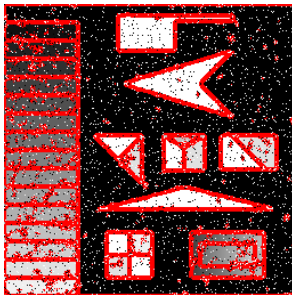


Figura: Borde

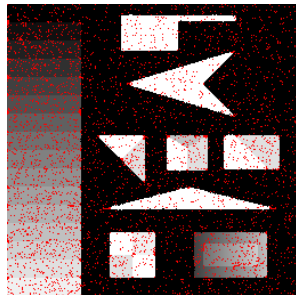


Figura: Esquina

Evaluación Susan Ruidos Sal y Pimienta

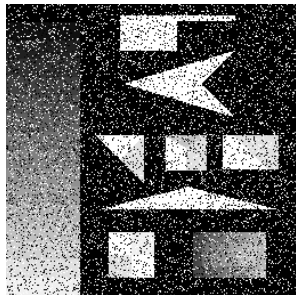


Figura: Imagen Ruidosa con 0.1.

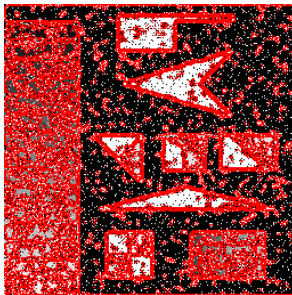


Figura: Borde

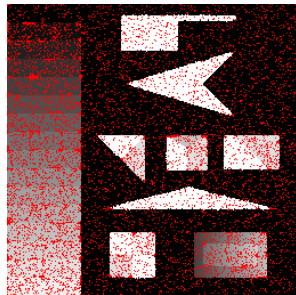


Figura: Esquina

Punto 3: Transformada de Hough para Rectas

Objetivo

Encontrar líneas, círculos o elipses mediante un espacio de parámetros y un sistema de votación.

Representación normal de una recta:

$$x \cos(\theta) + y \sin(\theta) = r$$

Algoritmo:

- 1 Detectar bordes y binarizar la imagen.
- 2 Discretizar el espacio (r, θ) y crear una matriz acumuladora $A(i, j)$.
- 3 Para cada píxel de borde (x_k, y_k) :

$$r = x_k \cos(\theta) + y_k \sin(\theta)$$

se incrementa $A(i, j)$.

- 4 Los máximos de A representan las rectas más votadas.

General 1

```
1 def transformada_de_hough():
2     global imagen_actual, imagen_operativa, umbral_hough
3
4     sigma = 1
5     t = int((2*sigma)+1)
6     ax = np.linspace(-(t - 1) / 2, (t - 1) / 2, t)
7     xx, yy = np.meshgrid(ax, ax)
8     kernel_init = np.exp(-(xx**2 + yy**2) / (2 * sigma**2))
9     imagen_ruidosa = fc.mascara(imagen_original, kernel_init, "
10     Gaussiano", grises=True, estandarizar=True, prewitt=False)
11     sobel_x = cv2.Sobel(imagen_ruidosa, cv2.CV_64F, 1, 0, ksize=3)
12     sobel_y = cv2.Sobel(imagen_ruidosa, cv2.CV_64F, 0, 1, ksize=3)
13     magnitud = np.hypot(sobel_x, sobel_y)
14     direccion = np.arctan2(sobel_y, sobel_x)
15     direccion = np.degrees(direccion)
16     magnitud_norm = cv2.normalize(magnitud, None, alpha=0, beta=255,
17     norm_type=cv2.NORM_MINMAX)
18     magnitud_norm = magnitud_norm.astype(np.float32)
```

General 2

```
1 umbral2 = fc.umbralizacion_Otsu(magnitud_norm,grises=True)[1]
2 umbral1 = 0.5 * umbral2
3 bordes = fc.bordes_canny(magnitud_norm,direccion,umbral1=int(umbral1),
    umbral2=int(umbral2))
4 imagen_operativa = bordes
5 mostrar_imagen(imagen_operativa, lblOutputImage)
6 root.update()
7 time.sleep(5)
8 imagen_operativa = fc.transformada_de_hough(bordes.astype(np.float32),
    imagen_original, umbral=umbral_hough)
9 imagen_operativa = imagen_operativa.astype(np.uint8)
10 mostrar_imagen(imagen_operativa, lblOutputImage)
11 set_status("Transformada de Hough aplicada a la imagen de trabajo.")
12
```

Paso 1, 2 y 3

```
1 def transformada_de_hough(imagen_actual, imagen_original, umbral):
2     alto, ancho = imagen_actual.shape[:2]
3     D = max(alto, ancho)
4     r_max = np.sqrt(2) * D
5     N_theta = 180
6     N_r = int(2 * r_max)
7     theta = np.deg2rad(np.linspace(-90, 90, N_theta, endpoint=False))
8     r = np.linspace(-r_max, r_max, N_r)
9     matriz_parametros = np.zeros((N_r, N_theta), dtype=np.float32)
10    y_blancos, x_blancos = np.nonzero(imagen_actual)
11    for x, y in zip(x_blancos, y_blancos):
12        for j, theta_j in enumerate(theta):
13            r_val = x * np.cos(theta_j) + y * np.sin(theta_j)
14            # buscar todos los r_i que cumplen la condicion
15            valid_r = np.where(np.abs(r - r_val) < epsilon)[0] # en
16            unidades de r
17            matriz_parametros[valid_r, j] += 1
18
```

Paso 4

```
1  # Dibujar las lineas detectadas
2  imagen_color = cv2.cvtColor(imagen_original, cv2.COLOR_GRAY2BGR)
3  indices = np.argwhere(matriz_parametros > umbral)
4
5  print(f'Se van a dibujar {len(indices)} rectas')
6  for r_i, i_theta in indices:
7      rho = r[r_i]
8      t = theta[i_theta]
9      a = np.cos(t)
10     b = np.sin(t)
11     x0 = a * rho
12     y0 = b * rho
13
14     # Puntos extremos de la recta
15     x1 = int(x0 + 1000 * (-b))
16     y1 = int(y0 + 1000 * (a))
17     x2 = int(x0 - 1000 * (-b))
18     y2 = int(y0 - 1000 * (a))
19
20     # Validar coordenadas antes de dibujar
```

Evaluación Hough

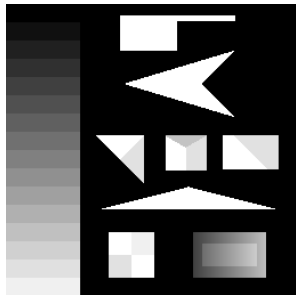


Figura: Imagen Original.
Todos con $\epsilon = 0.45$

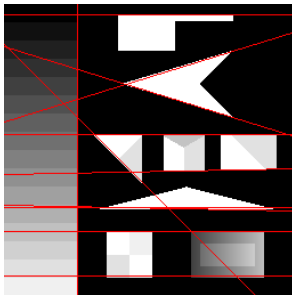


Figura: Umbral = 70

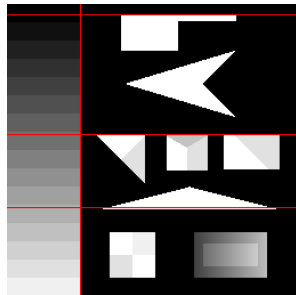


Figura: Umbral = 100

Evaluación Hough Ruidosa Gauss

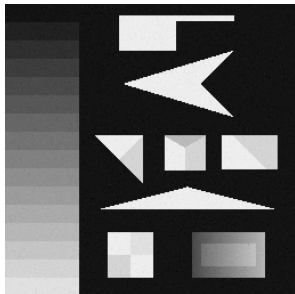


Figura: Imagen Ruidosa, 10 % de la imagen con 5 desviación.

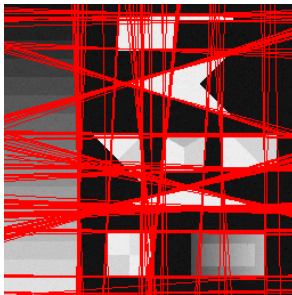


Figura: Umbral = 30

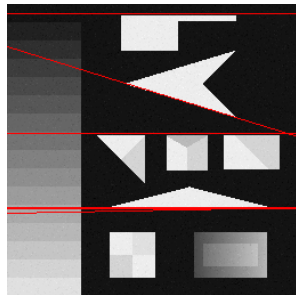


Figura: Umbral = 70

Evaluación Hough Ruidosa Gauss

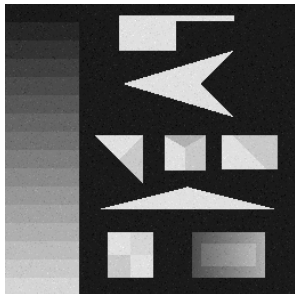


Figura: Imagen Ruidosa, 30 % de la imagen con 10 desviación.

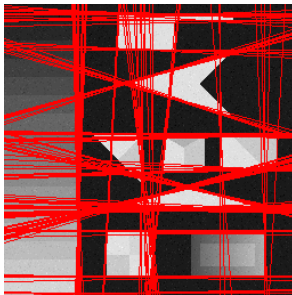


Figura: Umbral = 30

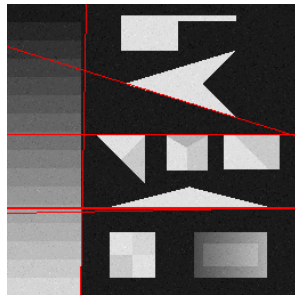


Figura: Umbral = 70

Evaluación Hough Ruidosa Gauss



Figura: Imagen Ruidosa, 50 % de la imagen con 20 desviación.

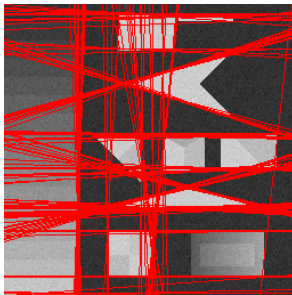


Figura: Umbral = 30

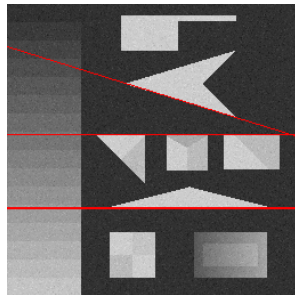


Figura: Umbral = 70

Evaluación Hough Ruidosa Exponencial

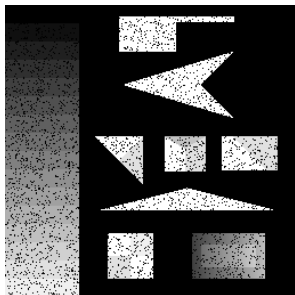


Figura: Imagen Ruidosa,
10 % de la imagen con
0.01.

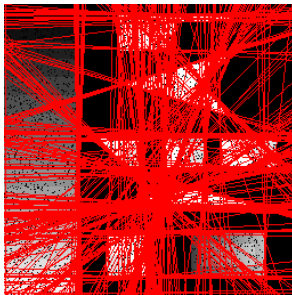


Figura: Umbral = 30

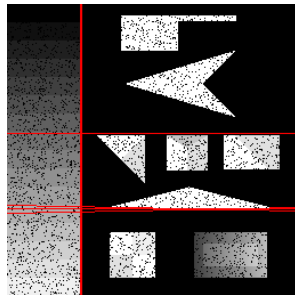


Figura: Umbral = 70

Evaluación Hough Ruidosa Exponencial

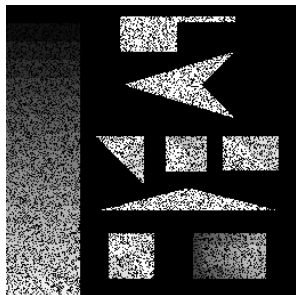


Figura: Imagen Ruidosa,
30 % de la imagen con
0.05.



Figura: Umbral = 30

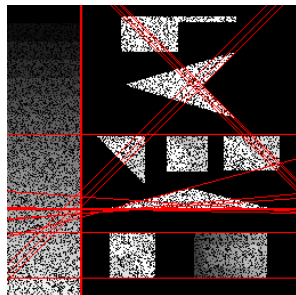


Figura: Umbral = 70

Evaluación Hough Ruidosa Exponencial

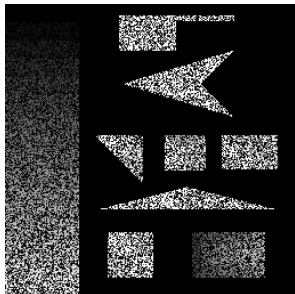


Figura: Imagen Ruidosa,
50 % de la imagen con
0.1.



Figura: Umbral = 30

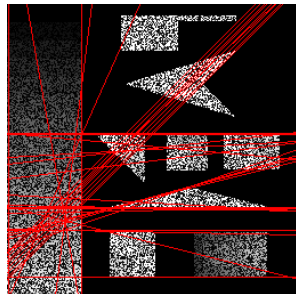


Figura: Umbral = 70

Evaluación Hough Ruidos Sal y Pimienta

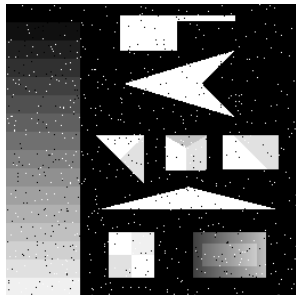


Figura: Imagen Ruidosa con 0.01.

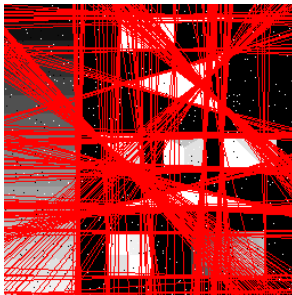


Figura: Umbral = 30

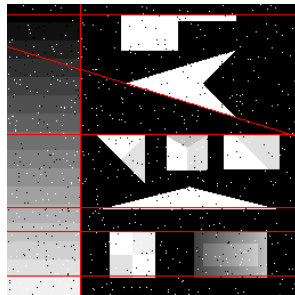


Figura: Umbral = 70

Evaluación Hough Ruidos Sal y Pimienta

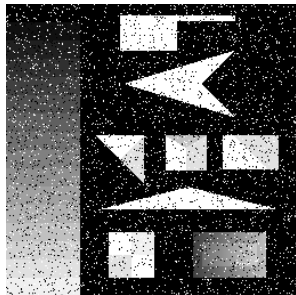


Figura: Imagen Ruidosa con 0.05.

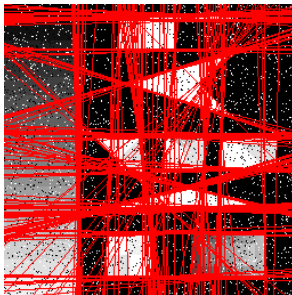


Figura: Umbral = 30

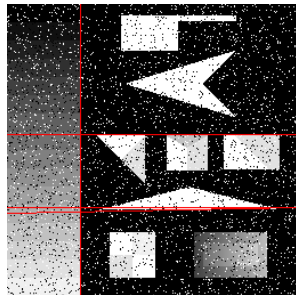


Figura: Umbral = 70

Evaluación Hough Ruidos Sal y Pimienta

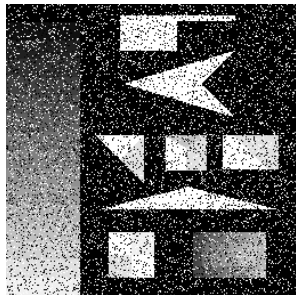


Figura: Imagen Ruidosa con 0.1.



Figura: Umbral = 30

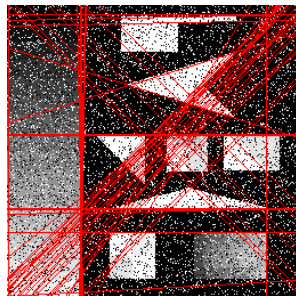


Figura: Umbral = 70

Punto 4: Segmentación por Intercambio de Píxeles

Concepto

Método alternativo a los contornos activos clásicos, más robusto y eficiente.

Características principales:

- Reduce el costo computacional.
- Puede combinarse con modelos estadísticos.
- Adecuado para aplicaciones en tiempo real.

Evolución de la Curva

Modelo:

- 1 El objeto y el fondo se describen con parámetros θ_1 y θ_0 .
- 2 Se definen listas L_{in} (borde interno) y L_{out} (borde externo).
- 3 Se evalúa:

$$F_d(x) = \log \left(\frac{\|\theta_0 - \theta(x)\|}{\|\theta_1 - \theta(x)\|} \right)$$

Si $F_d(x) > 0$, el píxel pertenece al objeto; si $F_d(x) < 0$, al fondo.

- La curva inicial se actualiza iterativamente moviendo píxeles entre L_{in} y L_{out} .
- La evolución se detiene al alcanzar la estabilidad o el número máximo de iteraciones.

General

```
1 def intercambio_de_pixeles():
2     global imagen_actual, imagen_operativa
3     promedios, matriz_final = seleccionar_regiones_y_rectangulo()
4     iteraciones = 10000 # Nmero máximo de iteraciones
5     for i in range(iteraciones):
6         set_status(f"Intercambio de pxeles: Iteracin {i+1} de {
7             iteraciones}")
8         matriz_anterior = matriz_final.copy()
9         matriz_final = fc.intercambio_de_pixeles(imagen_actual,
10            promedios, matriz_final)
11         imagen_operativa = fc.matriz_a_visual(imagen_actual,
12            matriz_final)
13         mostrar_imagen(imagen_operativa, lblOutputImage)
14         root.update()
15         time.sleep(0.25) # Pausa para visualizar el progreso
16     if np.array_equal(matriz_final, matriz_anterior):
17         break
```

```
1 def seleccionar_regiones_y_rectangulo():
2     global imagen_actual, imagen_operativa, roi_actual
3     regiones = []
4     rectangulo = None
5     promedios = []
6     seleccionando = False
7     x0, y0 = -1, -1
8
```

Seleccionar Promedios

```
1 def seleccionar_region(event, x, y, flags, param):
2     nonlocal seleccionando, x0, y0, regiones
3     if event == cv2.EVENT_LBUTTONDOWN:
4         seleccionando = True
5         x0, y0 = x, y
6     elif event == cv2.EVENT_MOUSEMOVE and seleccionando:
7         imagen_temp = imagen_mostrar.copy()
8         cv2.rectangle(imagen_temp, (x0, y0), (x, y), 255, 1)
9         cv2.imshow("Seleccionar regiones", imagen_temp)
10    elif event == cv2.EVENT_LBUTTONUP:
11        seleccionando = False
12        x1, y1 = x, y
13        regiones.append((x0, y0, x1, y1))
14        cv2.rectangle(imagen_mostrar, (x0, y0), (x1, y1), 255, 1)
15        cv2.imshow("Seleccionar regiones", imagen_mostrar)
16
```

Seleccionar Región

```
1 def seleccionar_rectangulo(event, x, y, flags, param):
2     nonlocal rectangulo, seleccionando, x0, y0
3     if event == cv2.EVENT_LBUTTONDOWN:
4         seleccionando = True
5         x0, y0 = x, y
6     elif event == cv2.EVENT_MOUSEMOVE and seleccionando:
7         imagen_temp = imagen_mostrar.copy()
8         cv2.rectangle(imagen_temp, (x0, y0), (x, y), 128, 2)
9         cv2.imshow("Seleccionar rectngulo", imagen_temp)
10    elif event == cv2.EVENT_LBUTTONUP:
11        seleccionando = False
12        rectangulo = (x0, y0, x, y)
13        cv2.rectangle(imagen_mostrar, (x0, y0), (x, y), 128, 2)
14        cv2.imshow("Seleccionar rectngulo", imagen_mostrar)
15
```

Seleccionador

```
1 imagen_mostrar = imagen_actual.copy()
2 cv2.imshow("Seleccionar regiones", imagen_mostrar)
3 cv2.setMouseCallback("Seleccionar regiones", seleccionar_region)
4 set_status("Selecciona dos regiones rectangulares con clic izquierdo y
   arrastre.")
5 while len(regiones) < 2:
6     cv2.waitKey(1)
7 cv2.destroyWindow("Seleccionar regiones")
8
9 for (x0, y0, x1, y1) in regiones:
10     x0, x1 = sorted([x0, x1])
11     y0, y1 = sorted([y0, y1])
12     region = imagen_actual[y0:y1, x0:x1]
13     promedio = float(region.mean())
14     promedios.append(promedio)
```

Seleccionador

```
1 seleccionando = False
2 x0, y0 = -1, -1
3 cv2.imshow("Seleccionar rectngulo", imagen_mostrar)
4 cv2.setMouseCallback("Seleccionar rectngulo", seleccionar_rectangulo)
5 set_status("Selecciona un rectngulo adicional arrastrando con el mouse
6         .")
7 while rectangulo is None:
8     cv2.waitKey(1)
9     cv2.destroyWindow("Seleccionar rectngulo")
10
11 x0, y0, x1, y1 = rectangulo
12 x0, x1 = sorted([x0, x1])
13 y0, y1 = sorted([y0, y1])
```


Armamos la matriz

```
1 h, w = imagen_actual.shape[:2]
2
3
4 matriz_final = np.full((h, w), 3, dtype=np.int8) # fondo = 3
5 grosor = 2
6 y0o, y1o = max(0, y0-2*grosor), min(h, y1+2*grosor)
7 x0o, x1o = max(0, x0-2*grosor), min(w, x1+2*grosor)
8 matriz_final[y0o:y1o, x0o:x1o] = 1
9
10
11 y0i, y1i = max(0, y0-grosor), min(h, y1+grosor)
12 x0i, x1i = max(0, x0-grosor), min(w, x1+grosor)
13 matriz_final[y0i:y1i, x0i:x1i] = -1
14
15 matriz_final[y0:y1, x0:x1] = -3
16
```

Intercambio de Píxeles

```
1 def intercambio_de_pixeles(imagen, promedios, matriz):
2
3     theta0, theta1 = promedios
4     h, w = imagen.shape
5     matriz = matriz.copy()
6
7     dist0 = np.abs(imagen - theta0) + 1e-6
8     dist1 = np.abs(imagen - theta1) + 1e-6
9     Fd = np.log(dist0 / dist1)
10
```

Paso 1

```
1 Lout_coords = np.argwhere(matriz == 1)
2     for x, y in Lout_coords:
3         if Fd[x, y] > 0:
4             matriz[x, y] = -1 # mover a Lin
5             for i, j in [(x-1,y),(x+1,y),(x,y-1),(x,y+1)]:
6                 if 0 <= i < h and 0 <= j < w:
7                     if matriz[i,j] == 3:
8                         matriz[i,j] = 1 # agregar a Lout
9
```

Paso 2

```
1 Lin_coords = np.argwhere(matriz == -1)
2     for x, y in Lin_coords:
3         vecinos = []
4         for i, j in [(x-1,y),(x+1,y),(x,y-1),(x,y+1)]:
5             if 0 <= i < h and 0 <= j < w:
6                 vecinos.append(matriz[i,j])
7                 if all(v != 3 and v != 1 for v in vecinos):
8                     matriz[x, y] = -3 # ahora interior
9
```

Paso 3

```
1 Lin_coords = np.argwhere(matriz == -1)
2     for x, y in Lin_coords:
3         if Fd[x, y] < 0:
4             matriz[x, y] = 1
5             for i, j in [(x-1,y),(x+1,y),(x,y-1),(x,y+1)]:
6                 if 0 <= i < h and 0 <= j < w:
7                     if matriz[i,j] == -3:
8                         matriz[i,j] = -1 # agregar a Lin
```

Paso 4

```
1 Lout_coords = np.argwhere(matriz == 1)
2     for x, y in Lout_coords:
3         vecinos = []
4             for i, j in [(x-1,y),(x+1,y),(x,y-1),(x,y+1)]:
5                 if 0 <= i < h and 0 <= j < w:
6                     vecinos.append(matriz[i,j])
7             if all(v != -3 and v != -1 for v in vecinos):
8                 matriz[x, y] = 3 # ahora exterior
```

Evaluación

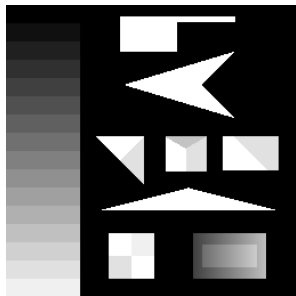


Figura: Imagen Original.

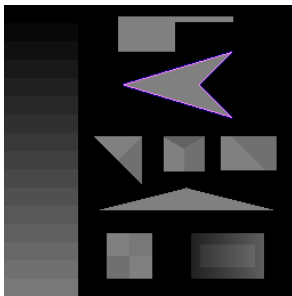


Figura: Intercambio

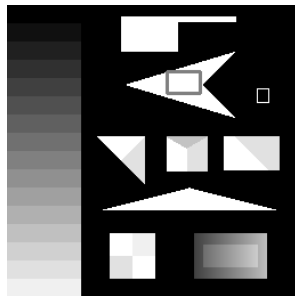


Figura: Fragmentos

Evaluación 2

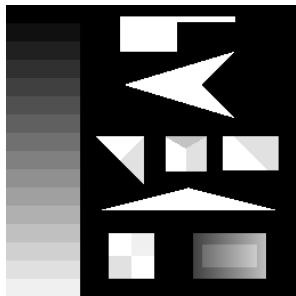


Figura: Imagen Original.

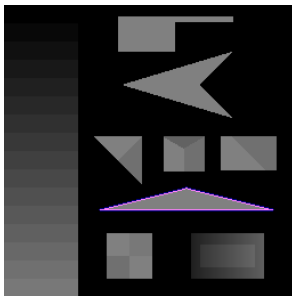


Figura: Intercambio

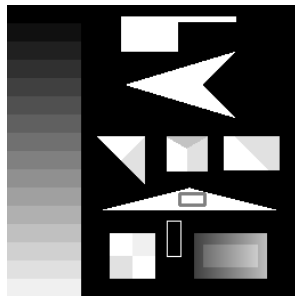


Figura: Fragmentos

Evaluación Ruidosa Gauss

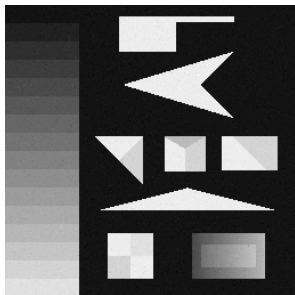


Figura: Imagen Ruidosa, 10 % de la imagen con 5 desviación.

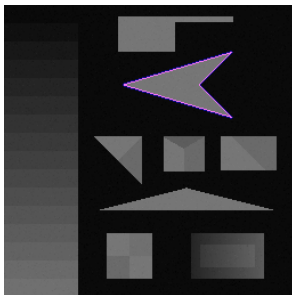


Figura: Intercambio

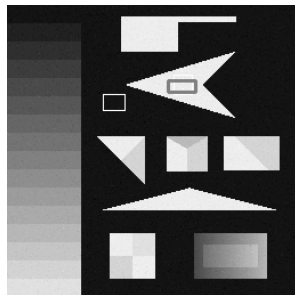


Figura: Fragmentos

Evaluación Ruidosa Gauss

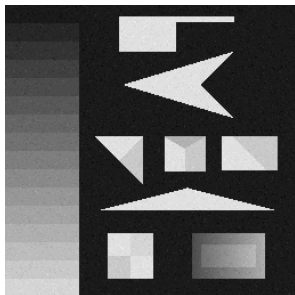


Figura: Imagen Ruidosa, 30 % de la imagen con 10 desviación.

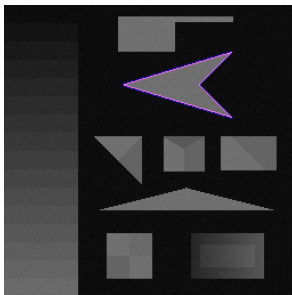


Figura: Intercambio

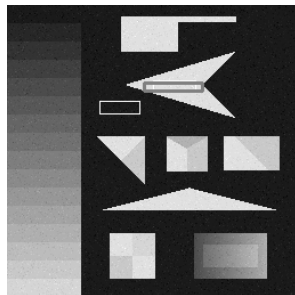


Figura: Fragmentos

Evaluación Ruidosa Gauss



Figura: Imagen Ruidosa, 50 % de la imagen con 20 desviación.

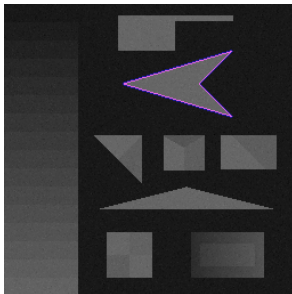


Figura: Intercambio

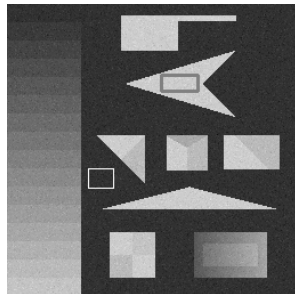


Figura: Fragmentos

Evaluación Ruidosa Exponencial

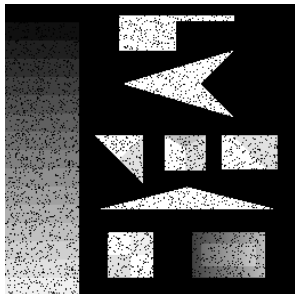


Figura: Imagen Ruidosa,
10 % de la imagen con
0.01.

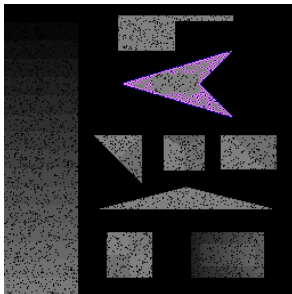


Figura: Intercambio

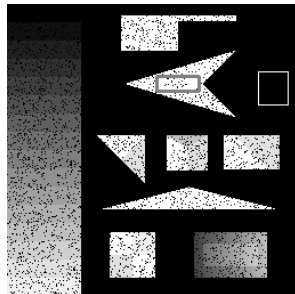


Figura: Fragmentos

Evaluación Ruidosa Exponencial

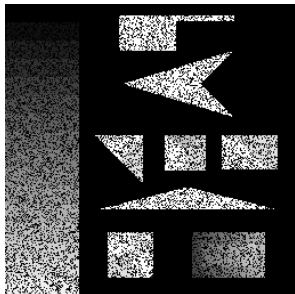


Figura: Imagen Ruidosa,
30 % de la imagen con
0.05.

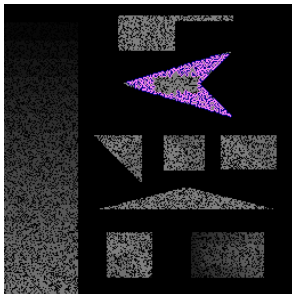


Figura: Intercambio

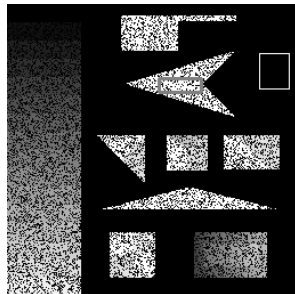


Figura: Fragmentos

Evaluación Ruidosa Exponencial

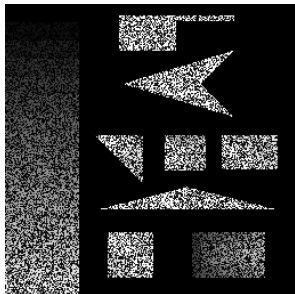


Figura: Imagen Ruidosa,
50 % de la imagen con
0.1.

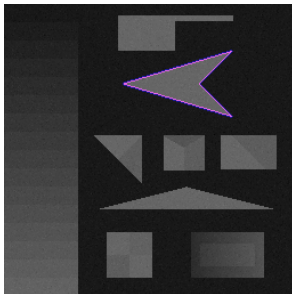


Figura: Intercambio

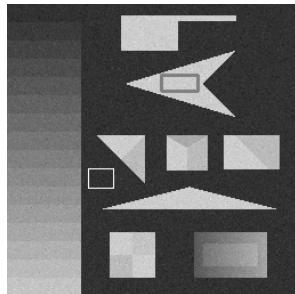


Figura: Fragmentos

Evaluación Ruidos Sal y Pimienta

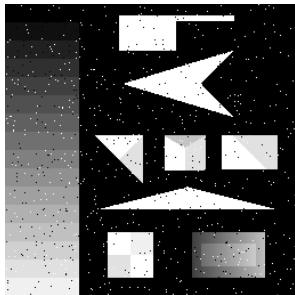


Figura: Imagen Ruidosa con 0.01.

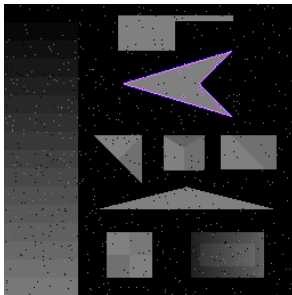


Figura: Intercambio

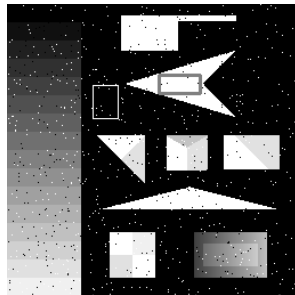


Figura: Fragmentos

Evaluación Ruidos Sal y Pimienta

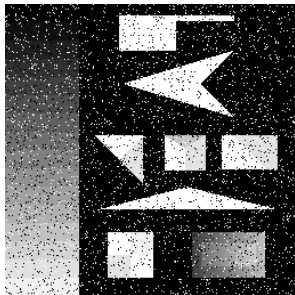


Figura: Imagen Ruidosa con 0.05.

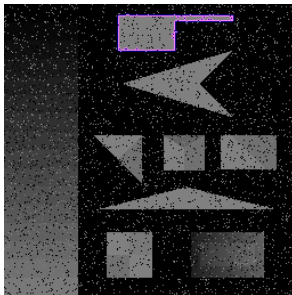


Figura: Intercambio

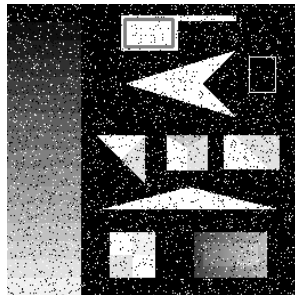


Figura: Fragmentos

Evaluación Ruidos Sal y Pimienta

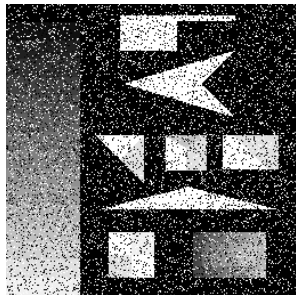


Figura: Imagen Ruidosa con 0.1.

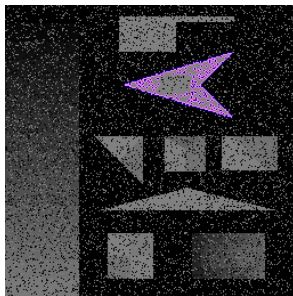


Figura: Intercambio

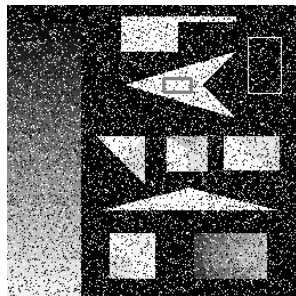


Figura: Fragmentos

Espero que haya gustado!!

